Aufzählungen (Enumerations)

Carsten Gips (HSBI)

TL;DR
Mit Hilfe von `enum` lassen sich Aufzählungstypen definieren (der Compiler erzeugt intern passende Klassen). Dabei wird den Konstanten eine fortlaufende Nummer zugeordnet, auf die mit `ordinal()` zugegriffen werden kann. Mit der Methode `values()` kann über die Konstanten iteriert werden, und mit `name()` kann eine Stringrepräsentation einer Konstanten erzeugt werden. Es sind keine Instanzen von Enum-Klassen erzeugbar, und die Enum-Konstanten sind implizit **`final`** und **`static`**.
Es lassen sich auch komplexe Enumerations analog zu Klassendefinition definieren, die eigene Konstruktoren, Felder und Methoden enthalten.
Videos

- VL Enumerations
- Demo Enumerations

---

1 Motivation

```java
public class Studi {
    public static final int IFM = 0;
    public static final int ELM = 1;
    public static final int ARC = 2;

    public Studi(String name, int credits, int studiengang) {
        // Wert für studiengang muss zwischen 0 und 2 liegen
        // Erwünscht: Konstanten nutzen
    }

    public static void main(String[] args) {
        Studi rainer = new Studi("Rainer", 10, Studi.IFM);
        Studi holger = new Studi("Holger", 3, 4);   // Laufzeit-Problem!
    }
}
```

Probleme:

- Keine Typsicherheit
- Konstanten gehören zur Klasse `Studi`, obwohl sie in anderem Kontext vermutlich auch interessant sind

Beispiel enums.v1.Studi (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/java-jvm/src/enums/v1/Studi.java)

2 Verbesserung: Einfache Aufzählung

```java
public enum Fach {
    IFM, ELM, ARC
}

```

```java

public class Studi {
    public Studi(String name, int credits, Fach studiengang) {
        // Typsicherheit für studiengang :-)
    }

    public static void main(String[] args) {
        Studi rainer = new Studi("Rainer", 10, Fach.IFM);
        Studi holger = new Studi("Holger", 3, 4);   // Syntax-Fehler!
    }
}
```

3 Einfache Aufzählungen: Eigenschaften

```java
public enum Fach {
    IFM, ELM, ARC
}
```

1. Enum-Konstanten (`IFM`, ...) sind implizit **`static`** und **`final`**
2. Enumerations (`Fach`) nicht instantiierbar
3. Enumerations stellen einen neuen Typ dar: hier der Typ `Fach`
4. Methoden: `name()`, `ordinal()`, `values()`, `toString()`

3.1 Wiederholung *static*

Attribute:

- **`static`** Attribute sind Eigenschaften/Zustände der **Klasse**
- Gelten in jedem von der Klasse erzeugten Objekt
- Unterschiedliche Lebensdauer:
  - Objektattribute (Instanzvariablen): ab **`new`** bis zum Garbage Collector
  - Statische Variablen: Laufzeitumgebung (JVM) lädt und initialisiert die Klasse (**`static`** Attribute existieren, bis die JVM die Klasse entfernt)

Methoden:

- **`static`** deklarierte Methoden sind **Klassenmethoden**
- Können direkt auf der Klasse aufgerufen werden
- Beispiele: `Math.max()`, `Math.sin()`, `Integer.parseInt()`
- **Achtung**: In Klassenmethoden nur Klassenattribute nutzbar (keine Instanzattribute!), d.h. keine **`this`**-Referenz nutzbar

3.2 Wiederholung *final*: Attribute/Methoden/Klassen nicht änderbar

- Attribute: **`final`** Attribute können nur einmal gesetzt werden

```java
void foo() {
    int i = 2;
    final int j = 3;
    final int k;
    i = 3;
    j = 4;  // Compilerfehler
    k = 5;
    k = 6;  // Compilerfehler
}
```

Beispiel enums.FinalDemo (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/java-jvm/src/enums/FinalDemo.java)

- Methoden: `final` deklarierte Methoden können bei Vererbung nicht überschrieben werden
- Klassen: von `final` deklarierten Klassen können keine Unterklassen gebildet werden

4 Einfache Aufzählungen: Eigenschaften (cnt.)

```java
// Referenzen auf Enum-Objekte können null sein
Fach f = null;
f = Fach.IFM;

// Vergleich mit == möglich
// equals() unnötig, da Vergleich mit Referenz auf statische Variable
if (f == Fach.IFM) {
    System.out.println("Richtiges Fach :-)");
}

// switch/case
switch (f) {
    case IFM:   // Achtung: *NICHT* Fach.IFM
        System.out.println("Richtiges Fach :-)");
        break;
    default:
        throw new IllegalArgumentException("FALSCHES FACH: " + f);
}
```

Außerdem können wir folgende Eigenschaften nutzen (u.a., s.u.):

- Enumerations haben Methode `String toString()` für die Konstanten
- Enumerations haben Methode `final T[] values()` für die Iteration über die Konstanten

Demo: enums.v2.Studi (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/blob/master/markdown/java-jvm/src/enums/v2/Studi.java)

5 Enum: Genauer betrachtet

```java
public enum Fach {  IFM, ELM, ARC  }
```

Compiler sieht (*in etwa*):

```
public class Fach extends Enum {
    public static final Fach IFM = new Fach("IFM", 0);
    public static final Fach ELM = new Fach("ELM", 1);
    public static final Fach ARC = new Fach("ARC", 2);

    private Fach( String s, int i ) { super( s, i ); }
}
```

=> **Singleton-Pattern** für Konstanten

6 Enum-Klassen: Eigenschaften

```
public enum Fach {
    IFM,
    ELM("Elektrotechnik Praxisintegriert", 1, 30),
    ARC("Architektur", 4, 40),
    PHY("Physik", 3, 10);

    private final String description;
    private final int number;
    private final int capacity;

    Fach() { this("Informatik Bachelor", 0, 60); }
    Fach(String descr, int number, int capacity) {
        this.description = descr;  this.number = number;  this.capacity =
            capacity;
    }
    public String getDescription() {
        return "Konstante: " + name() + " (Beschreibung: " + description
                + ", Kapazitaet: " + capacity + ", Nummer: " + number
                + ", Ordinal: " + ordinal() + ")";
    }
}
```

6.1 Konstruktoren und Methoden für Enum-Klassen definierbar

- Kein eigener Aufruf von `super` (!)
- Konstruktoren implizit `private`

6.2 Compiler fügt automatisch folgende Methoden hinzu (Auswahl):

- Strings:
  - `public final String name()` => Name der Konstanten (`final`!)
  - `public String toString()` => Ruft `name()` auf, überschreibbar
- Konstanten:

- `public final T[] values()` => Alle Konstanten der Aufzählung
- `public final int ordinal()` => Interne Nummer der Konstanten (Reihenfolge des Anlegens der Konstanten!)
- `public static T valueOf(String)` => Zum String passende Konstante (via `name()`)

Hinweis: Diese Methoden gibt es auch bei den "einfachen" Enumerationen (s.o.).

Demo: enums.v3 (https://github.com/Programmiermethoden-CampusMinden/PM-Lecture/tree/master/markdown/java-jvm/src/enums/v3/)

7 Wrap-Up

- Aufzählungen mit Hilfe von `enum` (Compiler erzeugt intern Klassen)
- Komplexe Enumerations analog zu Klassendefinition: Konstruktoren, Felder und Methoden (keine Instanzen von Enum-Klassen erzeugbar)
- Enum-Konstanten sind implizit **`final`** und **`static`**
- Compiler stellt Methoden `name()`, `ordinal()` und `values()` zur Verfügung
  - Name der Konstanten
  - Interne Nummer der Konstanten (Reihenfolge des Anlegens)
  - Array mit allen Konstanten der Enum-Klasse

8 Zum Nachlesen

- Oracle Corporation (2022)
- Ullenboom (2021, Kap. 6.4.3 und 10.7)

Lernziele

- k2: Vorgänge beim Initialisieren von Enum-Klassen (Hinweis: static)
- k3: Erstellung komplexer Enumerationen mit Feldern und Konstruktoren
- k3: Nutzung von name(), ordinal() und values() in Enum-Klassen

Quizzes

- Quiz Enumerations (ILIAS)

Quellen

Oracle Corporation. 2022. „The Java Tutorials“. 2022. https://docs.oracle.com/javase/tutorial/.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** abfbbeb (tooling: fix remaining issues, 2025-04-29)